두 회에 걸쳐 확인했다 —— 파라미터 목록의 능력이 이펙트를 보이게 한다는 것, 그리고 그것이 고차 함수를 살아남는다는 것. 하지만, 어느 해상도로 보이는가? 어떤 함수가 Database를 건네받았음을 알아도, 그것은 데이터베이스에 닿을 수 있음을 알릴 뿐이다 —— 그저 읽는지 쓰기도 하는지도, 데이터베이스를 독차지하고 싶은지 공유할 수 있는지도 알리지 않는다. 유무는 거친 결이다. 이번 회는, 이펙트에 더 고운 결을 주는 이야기이고, 그것을 어떻게 줄지를 정하는 하나의 껄끄러운 사례의 이야기다.

Logger 문제

두 점밖에 없는 시스템이 표현할 수 없는 사례가 이것이다:

```
fn read_users(db: &borrowed Database) -> Vec[User]       // 읽기
fn save_order(db: &mut borrowed Database) -> unit        // 쓰기, 배타
fn log_info(logger: &borrowed Logger, msg: &borrowed str)  // 쓰기 —— 하지만 공유 가능
```

앞의 둘은 편안하다. 사용자를 읽으려면 공유의 읽기 전용 핸들이 필요하고, 주문을 저장하려면 배타적 쓰기 접근이 필요하다. 그것들은, 대부분의 언어가 주는 두 대여 형태에 깔끔하게 대응한다: 공유 읽기의 &와, 배타 쓰기의 &mut.

셋째가 대응을 깨뜨린다. logger는 쓰인다 —— 로그는 줄을 덧붙인다 —— 그래서 읽기가 아니다. 하지만 배타적이고 싶지도 않다: logger의 요점은 바로, 많은 부르는 이가, 많은 스레드마저도, 동시에 쓰고, 안전성은 내부에서(락, 혹은 락프리 큐) 짊어지는 것이다. 그것을 &borrowed로 넘기면, 타입은 쓰기에 대해 슬며시 거짓말을 한다. &mut borrowed로 넘기면, 그것을 배타로 선언하여, 그 존재 이유인 병행 로그를 금해 버린다. 옳은 선택이 없다. 어휘에, logger가 무엇인지를 말할 말이 없기 때문이다.

Rust가 묶는 두 축

말이 없는 이유는, &와 &mut가 저마다 한 번에 두 일을 하고 있기 때문이다. &는 공유이자 읽기 전용을 뜻하고, &mut는 배타이자 쓰기 가능을 뜻한다. 두 독립된 물음 —— 이것을 여럿이 동시에 쥘 수 있는가? 그리고 대여자는 그것을 통해 변경할 수 있는가? —— 이, 하나의 양자택일로 융합되어 있다. 그 융합은 편리하고, 대부분의 코드를 덮지만, 네 조합 중 하나를 이름 없이 남긴다: 공유이자 쓰기 가능. 그것이 바로 logger 다.

Mere의 응답은, 연재가 몇 번이고 보여 온 습관에서 곧장 따른다: 하나의 이름이 슬며시 두 뜻을 나르고 있으면, 가른다. 그래서 대여 표기는 두 직교 축의 그리드가 된다 —— 공유성(shared / exclusive)과 조작(read / write) —— 그리고 네 점 저마다 이름이 붙는다:

표기	공유성	쓰기	예
&R T(기본)	공유	읽기만	Config, Users
&shared write R T	공유	가능	Logger, Cache, Metrics
&exclusive R T	배타	읽기만	(드물)

애초에 왜 타입에 싣는가

더 값싼 답도 있었다. 검토된 대안은, "단일 &를 유지하고, 무엇이 허용되는지는 각 능력의 API에 정하게 한다"에서, Logger<Write, Shared> 같은 마커 태그를 능력 타입에 붙이는 데까지 미쳤다. 단일 대여 선택지는 세우기가 가장 쉽다 —— 하지만 그것은, 병행성의 물음을 통째로 능력의 구현으로 밀어내고, 거기는 타입 시스템에 보이지 않는다: 공유 핸들 너머의 cache.set이 안전한지는, 컴파일러가 검사하는 사실이 아니라, 작성자가 하는 약속이 된다. 검사할 수 있는 것의 최대화를 전제의 전부로 삼는 언어에게, 병행 안전성을 미검사의 관례에 맡기는 것은 틀린 거래다. 그래서 그 선택지는 단적으로 기각되었다.

마커 태그 수법은, 그리드가 표현할 수 있는 모든 것과 그 이상을 표현할 수 있다 —— Io 태그도, Gpu 태그도, 무엇이든 발명할 수 있다 —— 하지만 그 일반성이야말로 문제다. 그것은 능력 선언을, 모든 타입 작성자가 올바르게 플레이해야 할 작은 태그 언어로 바꾸고, 네 소박한 조합이 실제로 필요로 하는 것보다 무겁다. Mere는 그리드를 주축으로 택하고, 조작 목록의 양식(Rust의 &self / &mut self 메서드)을 능력의 API가

런타임 비용: 제로

좁힘은 종이 위에서 이뤄졌고, 그다음 몇 개의 구현 슬라이스로 컴파일러에 떨어졌다 —— 그리고 가장 말해 주는 것은, 무엇이 바뀌지 않아도 됐는가다. 네 모드는 & 뒤의 문맥 의존 키워드이고, 타입 안을 borrow_mode로 실려 가며, 단일화는 그것들을 subtyping 없이 구별한다. 그래서 &mut R Database를 원하는 곳에 &R Database를 넘기는 것은, 호출 지점에서 잡히는 타입 에러다. 하지만 런타임과 세 backend(C·LLVM·Wasm)는 모드를 그저 무시한다: 대여는 모드와 상관없이 포인터이므로, 생성되는 코드는 동일하고, backend 간 feature parity는 코드 생성에 일절 손대지 않고 유지되었다.

이것은 뷰 타입이 Trivial인 것과 같은 형태다: 완전히 타입 시스템 안에 살고, 런타임에는 증발 하는 보장. &shared write와 &mut의 구별은, 프로그래머에게 시그니처에서의 얼마간의 정직함을 요구하고, 도는 프로그램에는 아무것도 요구하지 않는다. 동작하는 실례가 끝에서 끝까지 확인했다 —— 하나의 region 안에서 세 모드로 대여된 세 능력, 저마다 대여 너머로 쓸 수 있고, 어느 것도 서로 간섭하지 않는다. 그것은, 산문

병행 처리 장으로 미룬 것

&shared write에 이름을 붙이는 것은, 그것이 아직 답하지 않는 물음을 세운다: 그렇게 대여되기 위해, 타입은 무엇을 증명해야 하는가? 공유 쓰기 가능한 능력은 병행 접근에 대해 내부적으로 안전해야 하고, 타입이 그렇다고 선언할 수 있는 길이 있어야 한다 —— Send/Sync의 정신의 제약. 그리고 병행 backend로의 정확한 코드 생성 —— 락이 실제로 어디에 들어가는지, 쓰기 순서가 어떻게 보장되는지 —— 는, 생성할 병행 backend가 생길 때까지 남겨진다. 둘 다 일부러 미뤄진다. 병행 처리가, 예기되는 것이 아니라 현실이 되는, 이야기의 그 Part로.

이것으로 이펙트 모델은, 함수가 무엇에 닿아도 되는가뿐 아니라 어떻게 —— 읽기인지 쓰기인지, 단독인지 공유인지 —— 를 말할 수 있게 되었다. 남는 것은, 첫 이펙트 회가 깃발을 세운 인간공학의 문제다: 현실적인 함수는 여러 능력과 region을 요하고, 그 시그니처는 길어진다. 열린 물음은, 이펙트를 다시 보이지 않게 이동시키지 않으면서, 그것들을 어떻게 그룹화하는가였다. 다음 회: 능력의 묶음과, 그 라이프사이클 —— 시그니처의 spread, using 절, 그리고 메모리 모델로 묶어 되돌리는 with 구문.